Implement Client Script & UI Police (incident)

❖ project design phase phase

UI Policy: Use this first for simple, declarative field behaviours (making fields mandatory, read-only, or visible) without writing complex JavaScript. It executes on the client-side and handles UI logic natively.

Client Script (onChange / onLoad / onSubmit): Use this when you need advanced DOM manipulation, dynamic fetching via GlideAjax, or complex conditional calculations that a standard UI Policy condition builder cannot achieve.

❖ Sample Scenario & Implementation

Scenario:UI Policy Requirement: When the Incident State is set to Resolved, the Close code and Close notes fields must dynamically become Mandatory and Visible.

Step Client Script Requirement: Ensure that if State is Resolved, Close notes cannot be empty or contain fewer than 10 characters upon submission.

❖ Step-by-Code Implementation

A. UI Policy Configuration (Declarative Design)

Table: Incident (incident)

UI Policy Short Description: Make Closure Fields Mandatory on Resolution

Condition: State = Resolved (6)

Global: true

Reverse if true: true (Automatically clears/resets when state moves back to In Progress).

❖ UI Policy Actions:

Close code -> Mandatory: True | Visible: True

Close notes -> Mandatory: True | Visible: True

B. Client Script Implementation (onSubmit)

If your project requires advanced validation script files during the design build phase, use an onSubmit Client Script:

```
function onSubmit() {
   // Get the current state and close notes values
   Var state = g_form.getValue('incident_state');
   Var close_notes = g_form.getValue('close_notes');
    // Check if state is Resolved (Value: 6)
   Var RESOLVED_STATE = '6':
   If (state == RESOLVED_STATE) {
      If (!close_notes || closeNotes.trim().length < 10) {
         // Display an error message to the user
         G_form.showFieldMsg('close_notes', 'Please provide detailed Close Notes (minimum 10 characters) before resolving.', 'error');

         // Abort the form submission
```

```
        Return false;
      }
    }
}
```